

A Level Computer Science (9618)

Topical Past Paper Worksheet

Topic 4: Processor Fundamentals

Subtopic 4.3: Bit Manipulation

Concept 4.3.2: Bitwise Operations and Masking

Total Questions: 28

Mark Schemes begin on Page 50

Question 1 | Source: 9618_w25_qp_11 (Q4.b)

- 4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

- (b) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction XOR &12 is carried out.

--	--	--	--	--	--	--	--

[1]

Question 2 | Source: 9618_w25_qp_11 (Q4.c)

- 4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

- (c) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Write the contents of the ACC after the instruction AND #63 is carried out.

--	--	--	--	--	--	--	--

[1]

Question 3 | Source: 9618_w25_qp_11 (Q4.d)

- 4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

- (d) The ACC currently contains the following positive binary integer:

1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

The current contents of memory are:

Address	Data
98	00100100
99	00110001
100	00110011
101	10100011
102	10101100

Write the contents of the ACC after the instruction OR 100 is carried out.

--	--	--	--	--	--	--	--

[1]

Question 5 | Source: 9618_s25_qp_11 (Q8.b.i)

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (i) Write the bit manipulation instruction that can be used to set the least significant bit to 1 in an 8-bit register. All other bits must remain unchanged.

The instruction needs to work on a register that contains any 8-bit binary number.

..... [1]

Question 6 | Source: 9618_s25_qp_11 (Q8.b.ii)

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(ii) The ACC currently contains the following binary value.

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction XOR &FE is run.

--	--	--	--	--	--	--	--

[1]

Question 7 | Source: 9618_s25_qp_12 (Q7.b.i)

(b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(i) The ACC currently contains the following binary value.

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Write the result after the instruction OR B00001111 is run.

--	--	--	--	--	--	--	--

[1]

Question 8 | Source: 9618_s25_qp_12 (Q7.b.ii)

(b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(ii) The ACC currently contains the following binary value.

0	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

Write the result after the instruction XOR #30 is run.

--	--	--	--	--	--	--	--

[1]

Question 9 | Source: 9618_s25_qp_13 (Q5.b)

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system
ACC denotes Accumulator <address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Write the bit manipulation instructions that can be used to set only the most significant bit to 1 in an 8-bit register.

The instructions need to work on a register that contains any 8-bit binary number.

.....
 [2]

Question 10 | Source: 9618_w24_qp_12 (Q8.b.i)

- 8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current content of the ACC is shown:

ACC	1	0	0	1	1	0	1	0
------------	---	---	---	---	---	---	---	---

(i) The table has three sets of instructions. The binary number 10011010 is reloaded into the ACC before each set of instructions is run.

Complete the table by writing the content of the ACC after each set of instructions has run.

	Instructions	ACC content
1	LSL #2	
2	ADD #5 AND #30	
3	OR B11110010 INC ACC	

[3]

Question 11 | Source: 9618_w24_qp_12 (Q8.b.ii)

- 8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current content of the ACC is shown:

ACC	1	0	0	1	1	0	1	0
------------	---	---	---	---	---	---	---	---

- (ii) Explain how bit manipulation can be used to test whether the binary number stored in the ACC represents an odd denary number.

Write the bit manipulation instruction that will be used.

Explanation

.....

.....

.....

.....

Instruction

[3]

Question 12 | Source: 9618_w24_qp_13 (Q7.b)

- 7 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The instruction set also includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Explain how bit manipulation can be used to clear the data in an 8-bit register.

Write the bit manipulation instruction that will be used.

Explanation

.....

.....

.....

.....

Instruction

[3]

Question 13 | Source: 9618_s24_qp_11 (Q4.b)

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/ &n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/ &n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/ &n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
30	01110101
31	11111111
32	00000000
33	11001100
34	10101010

The current content of the ACC is shown:

1	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 10011010 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	AND 31	
2	XOR B01001111	
3	OR #30	

[3]

Question 14 | Source: 9618_s24_qp_12 (Q5.b)

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
25	11000110
26	11100001
27	10000001
28	11001101
29	00001111

The current content of the ACC is shown:

0	1	0	0	0	1	1	0
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each program has run.

The binary number 01000110 is reloaded into the ACC before each program is run.

Program number	Code	ACC content
1	XOR 29	
2	AND #29	
3	OR B11111111	

[3]

Question 15 | Source: 9618_s24_qp_13 (Q3.b)

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
100	00001101
101	10111110
102	11110011
103	00110111
104	00000000

The current content of the ACC is shown:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Complete the table by writing the content of the ACC after each instruction has run.

The binary number 11111111 is reloaded into the ACC before each instruction is run.

Instruction number	Instruction	ACC content
1	LSL #2	
2	XOR 100	
3	AND 103	

[3]

Question 16 | Source: 9618_w23_qp_11 (Q8.c.i)

(c) The following table shows another part of the instruction set for the same processor.

Instruction		Explanation
Opcode	Operand	
AND	Bn	Bitwise AND operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the operand
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
# denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B10100101

.....
.....

--	--	--	--	--	--	--	--

[1]

Question 17 | Source: 9618_w23_qp_11 (Q8.c.iii)

(c) The following table shows another part of the instruction set for the same processor.

Instruction		Explanation
Opcode	Operand	
AND	Bn	Bitwise AND operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the operand
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
# denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(iii) The current contents of the ACC are:

1	1	1	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00100101

.....
.....

--	--	--	--	--	--	--	--

[1]

Question 18 | Source: 9618_w22_qp_11 (Q6.b.i)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

XOR B00011111

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 19 | Source: 9618_w22_qp_11 (Q6.b.ii)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(ii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

AND B11110000

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 20 | Source: 9618_w22_qp_11 (Q6.b.iii)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(iii) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

OR B11001100

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 21 | Source: 9618_w22_qp_12 (Q7.b.i)

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(i) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

XOR B00011111

.....

--	--	--	--	--	--	--	--

[1]

Question 22 | Source: 9618_w22_qp_12 (Q7.b.ii)

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(ii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND 50

.....

--	--	--	--	--	--	--	--

[1]

Question 23 | Source: 9618_w22_qp_12 (Q7.b.iv)

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(iv) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR 51

.....

--	--	--	--	--	--	--	--

[1]

Question 24 | Source: 9618_w22_qp_13 (Q6.b.i)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(i) The current contents of the ACC are:

0	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

AND B01001100

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 25 | Source: 9618_w22_qp_13 (Q6.b.ii)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(ii) The current contents of the ACC are:

1	0	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

OR B01001111

.....

.....

--	--	--	--	--	--	--	--

[1]

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end

<address> can be an absolute or symbolic address
denotes a denary number, e.g. #123
B denotes a binary number, e.g. B01001101

The current contents of main memory are shown:

Address	Data
100	00001111
101	11110000
102	01010101
103	11111111
104	00000000

Each row of the following table shows the current contents of ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
11111111	OR 101	
00000000	XOR #15	
10101010	LSR #2	
01010101	AND 104	

[4]

Question 27 | Source: 9618_w21_qp_12 (Q8.b.i)

8 The Von Neumann model for a computer system uses registers.

(b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store the contents of ACC at the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123		

The current contents of main memory are shown:

Address	Data
100	01010101
101	11110000
102	00001111
103	00000000
104	11111111

- (i) In the following table, each row shows the current contents of the ACC in binary and the instruction that will be performed on those contents.

Complete the table by writing the new contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	New contents of the ACC
01010101	XOR 101	
11110000	AND 104	
00001111	LSL #4	
11111111	OR 102	

[4]

Question 28 | Source: 9618_s21_qp_12 (Q4.b.ii)

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

denotes a denary number, e.g. #123

The current contents of the main memory and selected values from the ASCII character set are:

Address Instruction

70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D

(b) Some bit manipulation instructions are shown in the table:

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The contents of the memory address 300 are shown:

Bit Number	7	6	5	4	3	2	1	0
300	0	1	1	0	0	1	1	0

- (ii) An assembly language program needs to test if bit number 2 in memory address 300 is a 1.

Complete the assembly language instruction to perform this test.

..... #4

[1]

Question 29 | Source: 9618_s21_qp_12 (Q4.b.iii)

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

denotes a denary number, e.g. #123

The current contents of the main memory and selected values from the ASCII character set are:

Address Instruction

70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D

(b) Some bit manipulation instructions are shown in the table:

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The contents of the memory address 300 are shown:

Bit Number	7	6	5	4	3	2	1	0
300	0	1	1	0	0	1	1	0

- (iii) An assembly language program needs to set bit numbers 4, 5, 6 and 7 to 0, but keep bits 0 to 3 with their existing values.

Write the assembly language instruction to perform this action.

.....
 [2]

MARK SCHEMES

Mark Scheme for Question 1 | Source: 9618_w25_ms_11 (Q4.b)

4(b)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	1	1	1	1	0	0	0	1	1
1	1	1	1	0	0	0	1			

Mark Scheme for Question 2 | Source: 9618_w25_ms_11 (Q4.c)

4(c)	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	0	0	1	0	0	0	1	1	1
0	0	1	0	0	0	1	1			

Mark Scheme for Question 3 | Source: 9618_w25_ms_11 (Q4.d)

4(d)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	0	1	1	1
1	1	1	1	0	0	1	1			

Mark Scheme for Question 4 | Source: 9618_w25_ms_13 (Q6.a)

6(a)	1 mark for each correct row <table border="1"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>Contents of the ACC after the execution of the instruction</th></tr> </thead> <tbody> <tr> <td>0000 1111</td><td>AND 101</td><td>0000 1110</td></tr> <tr> <td>0000 0000</td><td>LDM #100</td><td>0110 0100</td></tr> <tr> <td>0000 0001</td><td>XOR &F1</td><td>1111 0000</td></tr> <tr> <td>0001 0001</td><td>CMP 101</td><td>0001 0001</td></tr> </tbody> </table>	Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction	0000 1111	AND 101	0000 1110	0000 0000	LDM #100	0110 0100	0000 0001	XOR &F1	1111 0000	0001 0001	CMP 101	0001 0001	4
Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction															
0000 1111	AND 101	0000 1110															
0000 0000	LDM #100	0110 0100															
0000 0001	XOR &F1	1111 0000															
0001 0001	CMP 101	0001 0001															

Mark Scheme for Question 5 | Source: 9618_s25_ms_11 (Q8.b.i)

8(b)(i)	1 mark for: OR B0000 0001 // OR #1 // OR &1	1
---------	--	---

Mark Scheme for Question 6 | Source: 9618_s25_ms_11 (Q8.b.ii)

8(b)(ii)	1 mark for: 1010 1011	1
----------	--	---

Mark Scheme for Question 7 | Source: 9618_s25_ms_12 (Q7.b.i)

7(b)(i)	1 mark for: 1111 1111	1
---------	-------------------------------------	----------

Mark Scheme for Question 8 | Source: 9618_s25_ms_12 (Q7.b.ii)

7(b)(ii)	1 mark for: 0000 0011	
----------	-------------------------------------	--

Mark Scheme for Question 9 | Source: 9618_s25_ms_13 (Q5.b)

5(b)	1 mark for each correct instruction e.g. AND B00000000 / #0 / &0 OR B10000000 / #128 / &80	2
------	--	----------

Mark Scheme for Question 10 | Source: 9618_w24_ms_12 (Q8.b.i)

8(b)(i)	1 mark each correct content of ACC <table border="1"> <thead> <tr> <th></th><th>Instructions</th><th>ACC content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LSL #2</td><td>0110 1000</td></tr> <tr> <td>2</td><td>ADD #5 AND #30</td><td>0001 1110</td></tr> <tr> <td>3</td><td>OR B11110010 INC ACC</td><td>1111 1011</td></tr> </tbody> </table>		Instructions	ACC content	1	LSL #2	0110 1000	2	ADD #5 AND #30	0001 1110	3	OR B11110010 INC ACC	1111 1011	3
	Instructions	ACC content												
1	LSL #2	0110 1000												
2	ADD #5 AND #30	0001 1110												
3	OR B11110010 INC ACC	1111 1011												

Mark Scheme for Question 11 | Source: 9618_w24_ms_12 (Q8.b.ii)

8(b)(ii)	1 mark each to max 2 for explanation - An odd binary number will have a 1 in the Least Significant Bit (LSB) - A bit manipulation operation is required to access/mask only the LSB and clear all the others - Compare the result of the masking with denary 1 ... the result of the comparison will be true if the number is odd 1 mark for correct instruction - AND B00000001 // AND #1 // AND &01	3
----------	---	----------

Mark Scheme for Question 12 | Source: 9618_w24_ms_13 (Q7.b)

7(b)	1 mark for each bullet point for the explanation 1 mark for correct instruction - A bit manipulation operation is required to set all the bits to zero - Compare the result of the masking with 0 ... the result of comparison will be true if the register is cleared - AND B00000000 / #00 / &00	3
------	---	----------

Mark Scheme for Question 13 | Source: 9618_s24_ms_11 (Q4.b)

4(b)	1 mark for each correct answer: <table border="1"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>AND 31</td><td>1001 1010</td></tr> <tr> <td>2</td><td>XOR B01001111</td><td>1101 0101</td></tr> <tr> <td>3</td><td>OR #30</td><td>1001 1110</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	AND 31	1001 1010	2	XOR B01001111	1101 0101	3	OR #30	1001 1110	3
Program Number	Code	ACC Content												
1	AND 31	1001 1010												
2	XOR B01001111	1101 0101												
3	OR #30	1001 1110												

Mark Scheme for Question 14 | Source: 9618_s24_ms_12 (Q5.b)

5(b)	1 mark for each correct answer: <table border="1"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>XOR 29</td><td>0100 1001</td></tr> <tr> <td>2</td><td>AND #29</td><td>0000 0100</td></tr> <tr> <td>3</td><td>OR B11111111</td><td>1111 1111</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	XOR 29	0100 1001	2	AND #29	0000 0100	3	OR B11111111	1111 1111	3
Program Number	Code	ACC Content												
1	XOR 29	0100 1001												
2	AND #29	0000 0100												
3	OR B11111111	1111 1111												

Mark Scheme for Question 15 | Source: 9618_s24_ms_13 (Q3.b)

3(b)	1 mark for each correct answer: <table border="1"> <thead> <tr> <th>Instruction Number</th><th>Instruction</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LSL #2</td><td>1111 1100</td></tr> <tr> <td>2</td><td>XOR 100</td><td>1111 0010</td></tr> <tr> <td>3</td><td>AND 103</td><td>0011 0111</td></tr> </tbody> </table>	Instruction Number	Instruction	ACC Content	1	LSL #2	1111 1100	2	XOR 100	1111 0010	3	AND 103	0011 0111	3
Instruction Number	Instruction	ACC Content												
1	LSL #2	1111 1100												
2	XOR 100	1111 0010												
3	AND 103	0011 0111												

Mark Scheme for Question 16 | Source: 9618_w23_ms_11 (Q8.c.i)

8(c)(i)	0000 0101	1
---------	-----------	---

Mark Scheme for Question 17 | Source: 9618_w23_ms_11 (Q8.c.iii)

8(c)(iii)	1101 0010	1
-----------	-----------	---

Mark Scheme for Question 18 | Source: 9618_w22_ms_11 (Q6.b.i)

6(b)(i)	1000 1100	1
---------	-----------	---

Mark Scheme for Question 19 | Source: 9618_w22_ms_11 (Q6.b.ii)

6(b)(ii)	1001 0000	1
----------	-----------	---

Mark Scheme for Question 20 | Source: 9618_w22_ms_11 (Q6.b.iii)

6(b)(iii)	1101 1111	1
-----------	-----------	---

Mark Scheme for Question 21 | Source: 9618_w22_ms_12 (Q7.b.i)

7(b)(i)	0100 1100	1
---------	-----------	---

Mark Scheme for Question 22 | Source: 9618_w22_ms_12 (Q7.b.ii)

7(b)(ii)	0100 0001	1
----------	-----------	---

Mark Scheme for Question 23 | Source: 9618_w22_ms_12 (Q7.b.iv)

7(b)(iv)	1101 1111	1
----------	-----------	---

Mark Scheme for Question 24 | Source: 9618_w22_ms_13 (Q6.b.i)

6(b)(i)	0000 0100	1
---------	-----------	---

Mark Scheme for Question 25 | Source: 9618_w22_ms_13 (Q6.b.ii)

6(b)(ii)	1101 1111	1
----------	-----------	---

Mark Scheme for Question 26 | Source: 9618_w21_ms_11 (Q6.b)

6(b)	1 mark for each correct row	4															
<table border="1"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr> </thead> <tbody> <tr> <td>11111111</td><td>OR 101</td><td>11111111</td></tr> <tr> <td>00000000</td><td>XOR #15</td><td>00001111</td></tr> <tr> <td>10101010</td><td>LSR #2</td><td>00101010</td></tr> <tr> <td>01010101</td><td>AND 104</td><td>00000000</td></tr> </tbody> </table>			Current contents of the ACC	Instruction	New contents of the ACC	11111111	OR 101	11111111	00000000	XOR #15	00001111	10101010	LSR #2	00101010	01010101	AND 104	00000000
Current contents of the ACC	Instruction	New contents of the ACC															
11111111	OR 101	11111111															
00000000	XOR #15	00001111															
10101010	LSR #2	00101010															
01010101	AND 104	00000000															

Mark Scheme for Question 27 | Source: 9618_w21_ms_12 (Q8.b.i)

8(b)(i)	1 mark for each correct answer	4															
<table border="1"> <thead> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>New contents of the ACC</th></tr> </thead> <tbody> <tr> <td>01010101</td><td>XOR 101</td><td>1010 0101</td></tr> <tr> <td>11110000</td><td>AND 104</td><td>1111 0000</td></tr> <tr> <td>00001111</td><td>LSL #4</td><td>1111 0000</td></tr> <tr> <td>11111111</td><td>OR 102</td><td>1111 1111</td></tr> </tbody> </table>			Current contents of the ACC	Instruction	New contents of the ACC	01010101	XOR 101	1010 0101	11110000	AND 104	1111 0000	00001111	LSL #4	1111 0000	11111111	OR 102	1111 1111
Current contents of the ACC	Instruction	New contents of the ACC															
01010101	XOR 101	1010 0101															
11110000	AND 104	1111 0000															
00001111	LSL #4	1111 0000															
11111111	OR 102	1111 1111															

Mark Scheme for Question 28 | Source: 9618_s21_ms_12 (Q4.b.ii)

4(b)(ii)	AND	1
----------	-----	----------

Mark Scheme for Question 29 | Source: 9618_s21_ms_12 (Q4.b.iii)

4(b)(iii)	1 mark for AND, 1 mark for #15 AND #15	2
-----------	---	----------